Operating Systems: Concurrency, Synchronization & Deadlock Prevention

Race Conditions, Mutual Exclusion, Semaphores, Monitors, and Banker's Safety Algorithm

Doc Id: OS-SYNC-2024-003 | **Practice:** Concurrency, Locks & Deadlocks | **System:** POSIX Mutexes, Linux Futex, Dijkstra Semaphores | **Year:** 2024

1. Critical Section Problem and Synchronization Criteria

A critical section is a sequence of instructions accessing shared state that must not be concurrently executed by more than one thread.

A valid solution to the critical section problem must satisfy three mandatory criteria:

1. **Mutual Exclusion:** If process P is executing in its critical section, no other processes can execute in their critical sections.

2. **Progress:** If no process is executing in its critical section and some processes wish to enter, only those processes not executing in their remainder sections can participate in deciding who enters next, and this selection cannot be postponed indefinitely.

3. **Bounded Waiting:** There must exist a bound on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter and before that request is granted.

> *Peterson's Algorithm: Provides a mathematical, software-only mutual exclusion guarantee for two processes using shared variables 'flag[2]' and 'turn'. On modern out-of-order superscalar CPUs, hardware memory barriers (MFENCE) are required to prevent speculative instruction reordering from violating mutual exclusion.*

2. Hardware Synchronization Primitives vs. Software Primitives

Modern multiprocessor systems provide atomic hardware instructions that execute atomically across cache-coherent memory:

Primitive	Level	Blocking Behavior	Best Use Case	Overhead / Notes
Test-and-Set / CAS	Hardware instruction	Non-blocking / Atomic	Lock-free algorithms & spinlocks	x86 LOCK CMPXCHG; cache line contention
Spinlock	Kernel / Low-level	Busy-waits (spins on CPU core)	Short critical sections (< 1 µs)	Wastes CPU cycles; illegal in user space on single-core
Mutex (Mutual Exclusion)	OS / Library	Blocks thread (sleeps in runqueue)	General critical section protection	Context switches sleeping thread; Linux futex optimized
Counting Semaphore	OS primitive	Integer counter with wait/signal	Resource pool of N units	Dijkstra P() and V() operations; general signaling
Binary Semaphore	OS primitive	Restricted integer (0 or 1)	Mutual exclusion / Signaling	Differs from mutex: semaphore has no ownership
Condition Variable	Language / Library	Releases lock & sleeps atomically	Event notification / Producer-Consumer	Must always be paired with a mutex in a while loop

3. Deadlocks: The Four Coffman Conditions

A deadlock is a state where a set of processes are blocked because each process holds a resource and waits for another resource held by another process in the same set.

A deadlock can arise if and only if all four **Coffman Conditions** hold simultaneously:

• **1. Mutual Exclusion:** At least one resource must be held in a non-shareable mode.

• **2. Hold and Wait:** A process must be concurrently holding at least one resource and requesting additional resources held by other processes.

• **3. No Preemption:** Resources cannot be forcibly preempted from a process holding them; they can only be released voluntarily.

• **4. Circular Wait:** A closed chain of processes exists: P_0 waits for resource held by P_1, P_1 waits for P_2, ..., P_k waits for P_0.

4. Deadlock Handling Strategies and Banker's Algorithm

Operating systems manage deadlocks via three primary strategies: Deadlock Prevention, Deadlock Avoidance, and Deadlock Detection & Recovery (or the Ostrich algorithm of ignoring it).

Banker's Algorithm (Edsger Dijkstra): An avoidance algorithm testing for safe states before granting resource requests.

• Let Available[m] be available instances of resource types, Max[n][m] be max demand, Allocation[n][m] be currently held, and Need[i][j] = Max[i][j] - Allocation[i][j].

• A state is **SAFE** if there exists a safe execution sequence such that for each P_i, the resources that P_i can still request can be satisfied by current Available plus resources held by all preceding processes.

Strategy	Mechanism	Pros	Cons	Adoption in Modern OS
Deadlock Prevention	Disallow 1 of 4 Coffman conditions (e.g. strict global resource numbering)	Guarantees no deadlocks	Low resource utilization, restrictive	Strict lock ranking used in Linux kernel
Deadlock Avoidance	Banker's Algorithm: evaluate safety on every allocation request	Dynamic, flexible allocation	Requires knowing future maximum requests	Impractical for general-purpose OS
Detection & Recovery	Wait-For Graph cycle detection; abort thread or preempt resource	Optimistic; allows high resource usage	Recovery aborts transaction or causes data loss	Standard in database systems (DBMS)
Ostrich Algorithm	Ignore problem entirely; rely on user/watchdog restart	Zero runtime overhead	System may freeze if deadlock occurs	Standard approach in Linux, Windows, macOS